

Project 1 : TreeSize

Assignment 3: Documentation

Dated: 21.10.2025

Contents

1	Scope and Goal of the Project: TreeSize	3
2	User Documentation	3
2.1	How to Launch	3
2.1.1	Windows:	3
2.1.2	MacOS:	3
2.2	View on Launch	4
2.3	Scan	4
2.4	Cancel Scan	5
2.5	Search	5
2.6	Filter	6
2.7	Export	7
2.8	Graphical View On/Off	9
2.9	Text View Settings	9
2.10	Graphical Settings	9
2.11	Re Scan	10
2.12	DiskOver	10
2.13	Progress Bar	11
2.14	Full Path	11
2.15	Delete	12
2.16	Rename	13
2.17	Open File/Folder	14
2.18	1 Level Pie Chart	14
3	Implementation	15
3.1	Scanning	16
3.1.1	Requirements and how we analysed it	16
3.1.2	Code Explanation	16
3.2	Text and Context Menu	22
3.2.1	Requirements and how we analyzed it	22
3.2.2	Code Explanation	22
3.3	Tool Bar and Progress Bar	30
3.3.1	Requirements and how we analysed it	30
3.3.2	Code Explanation	30
3.4	Graphical View	34

3.4.1	Requirements and how we analysed it	34
3.4.2	Code Explanation	35
3.5	DiskOver	39
3.5.1	Requirements and how we analysed it	39
3.5.2	Code Explanation	39
4	Testing Plan	42
4.1	Objectives	42
4.2	Test Environment and Build Notes	43
4.3	Test Data	43
4.4	Detailed Test Cases	43
4.4.1	A. Scanning (SCN-xxx)	44
4.4.2	B. Tool Bar (TBR-xxx)	45
4.4.3	C. Graphical View (GPH-xxx)	45
4.4.4	D. DiskOver / Free Space (DSK-xxx)	46
4.5	Execution Priorities	46
5	Further Improvements	46
6	References	47
7	Declarations	47

1 Scope and Goal of the Project: TreeSize

This project aims to provide an efficient and intuitive way to visualize disk usage and manage storage space. Traditional file explorers show files and folders hierarchically but do not clearly indicate how much space each occupies. The TreeSize application overcomes this limitation by offering a graphical view of directory sizes, helping users quickly identify large files or folders, clean up unused data, and better understand their storage distribution.

2 User Documentation

2.1 How to Launch

This Application is compatible on Windows and MacOS Systems.

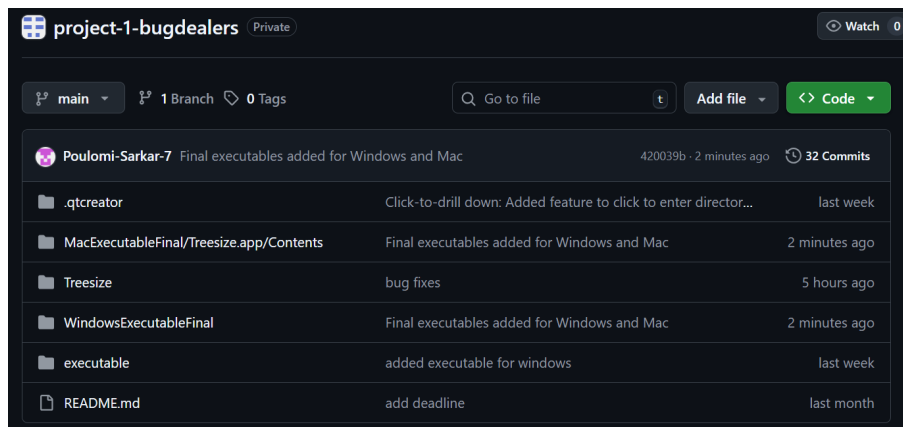


Figure 1: Git Repo

2.1.1 Windows:

To launch on windows:

There is a folder on the Git Repository called WindowsExecutableFinal which contains Treesize.exe

Follow the path: WindowsExecutableFinal\Treesize.exe

2.1.2 MacOS:

To launch on Mac Systems:

There is a folder on the Git Repository called MacExecutableFinal which contains Tree-size.app, which further contains Contents, which contains MacOS and that contains the final executable called Treesize

Follow the path: MacExecutableFinal\Treesize.app\Contents\MacOS\Treesize

2.2 View on Launch

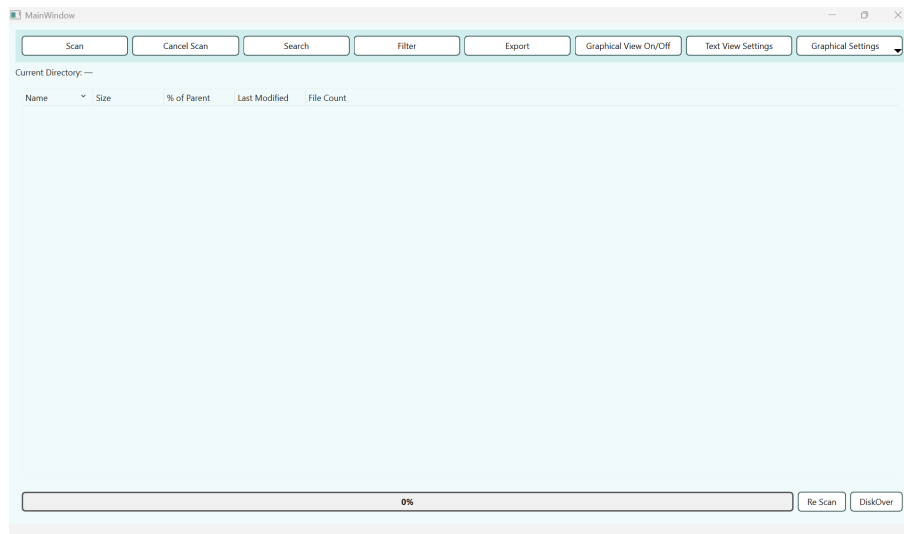


Figure 2: View on Launching

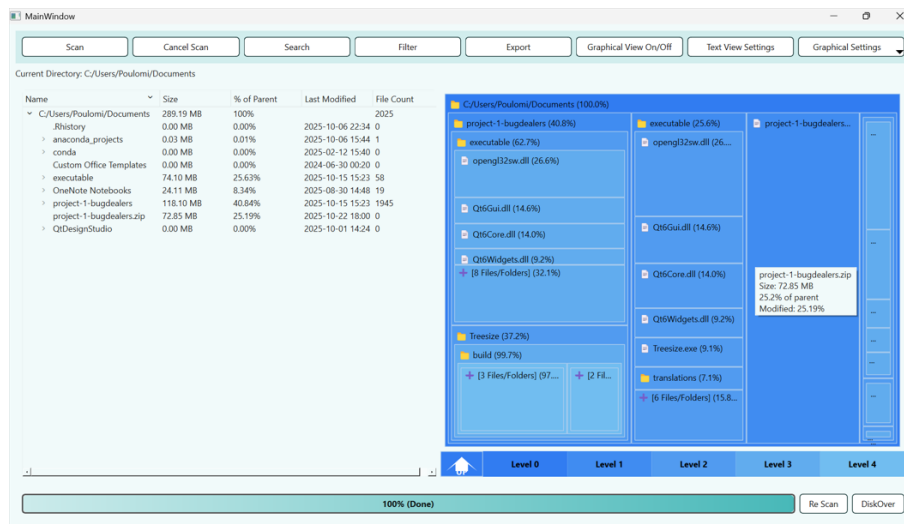


Figure 3: Complete View with Graphical Representation

2.3 Scan

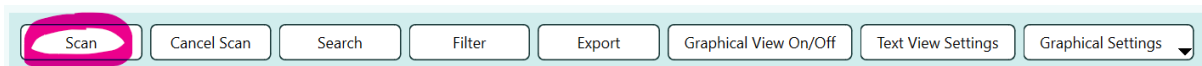


Figure 4: Scan Button

Clicking on this opens the file explorer of the device and allows the user to choose a directory they want to scan to see the files and folders contained in that particular directory. The progress bar updates showing percentage scanned and time elapsed as the scan is in progress.

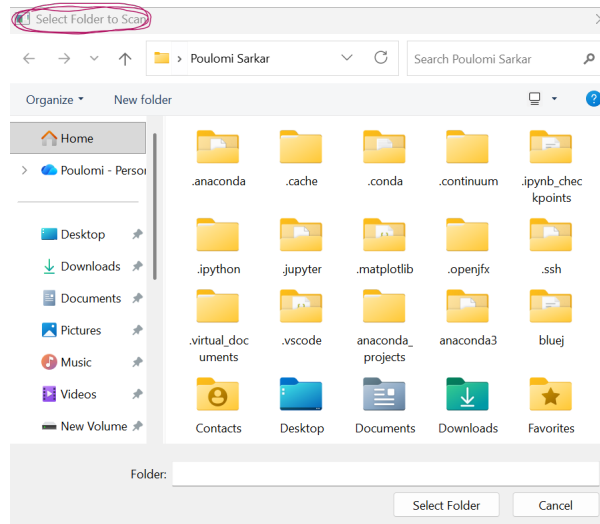


Figure 5: File Explorer View to choose directory to scan

2.4 Cancel Scan

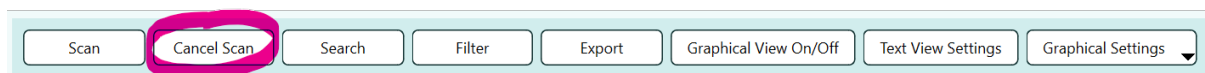


Figure 6: Cancel Scan Button

Clicking on this stops an ongoing directory scan and halts any further file processing. If clicked on after a completed scan, it resets the view to the initial view with no scanned directory and hence no text view. It also resets the progress bar to 0%

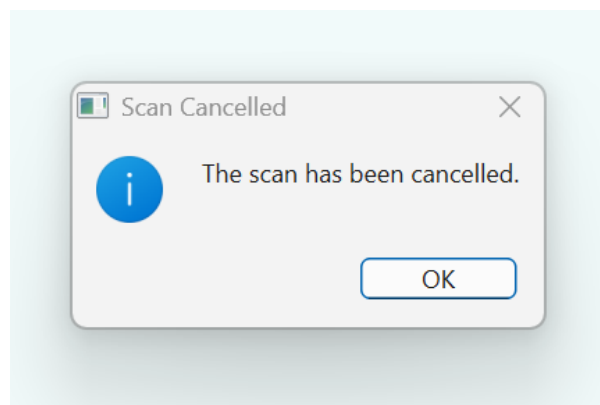


Figure 7: Message after canceling the scan

2.5 Search

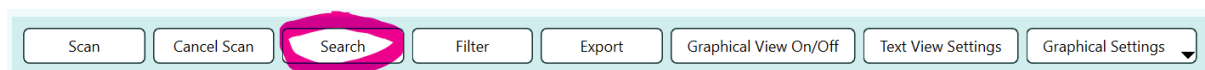


Figure 8: Search Button

Clicking on this allows the user to search for specific files or folders within the scanned directory tree. It collapses any root folders inside which a match has been found, so that the user can see where the search has matched. It also highlights the matched files or folders in yellow. It uses a non case sensitive substring match for this purpose.

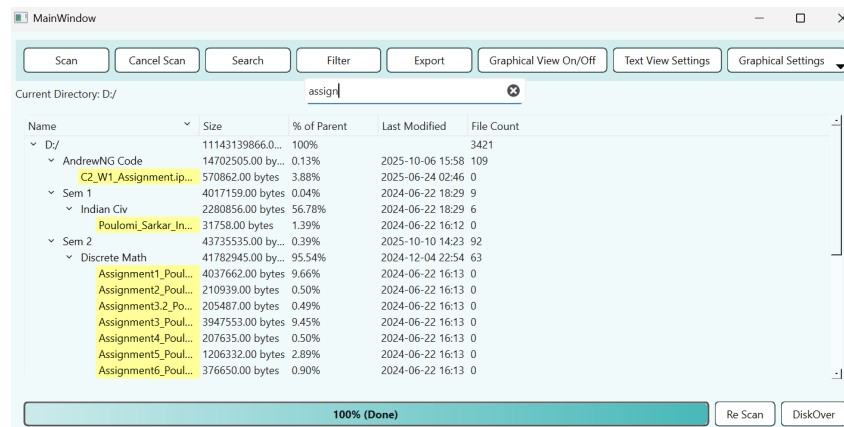


Figure 9: Search Example with "assign"

2.6 Filter

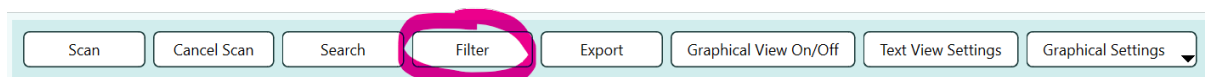


Figure 10: Filter Button

Clicking on this gives the user options to filter by Size (Greater than or Less than a particular size in the current chosen format - Bytes, KB, MB or GB), By count (Greater than or Less than a particular number of file counts), By format (e.g. .py or .png etc. It automatically gives a dropdown list of format options in that particular scanned directory), and By Type (images, documents etc.). For all filter matches it collapses the folders so that the filter match is displayed in bold, making it easier for the user to identify which files/folders have been filtered. The reset filter option, resets the view to the original scanned directory view.

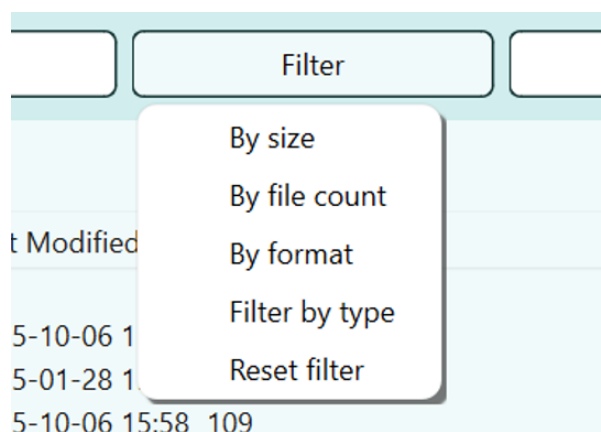


Figure 11: Filter Options

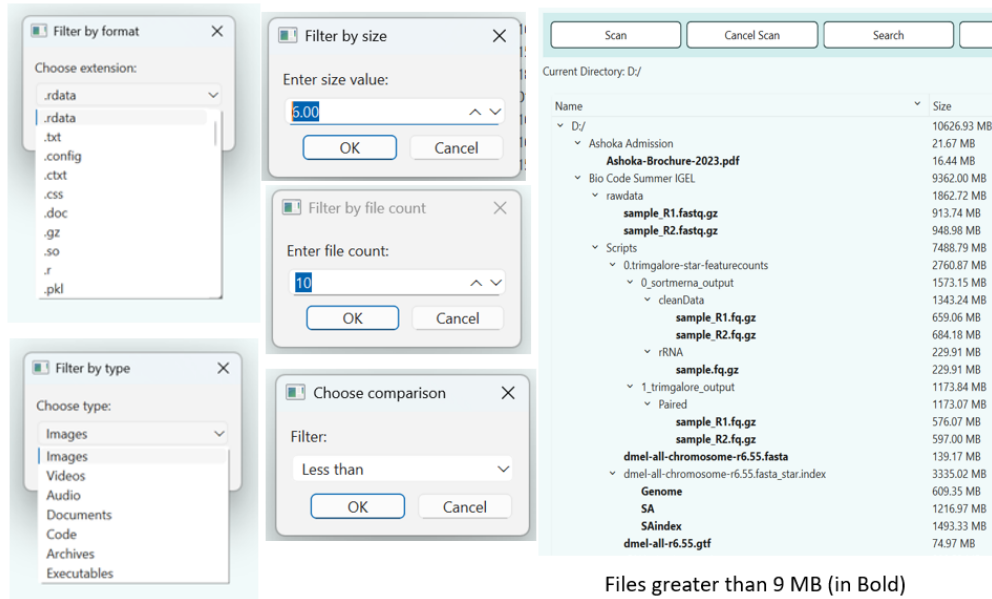


Figure 12: Filter Examples

2.7 Export



Figure 13: Export Button

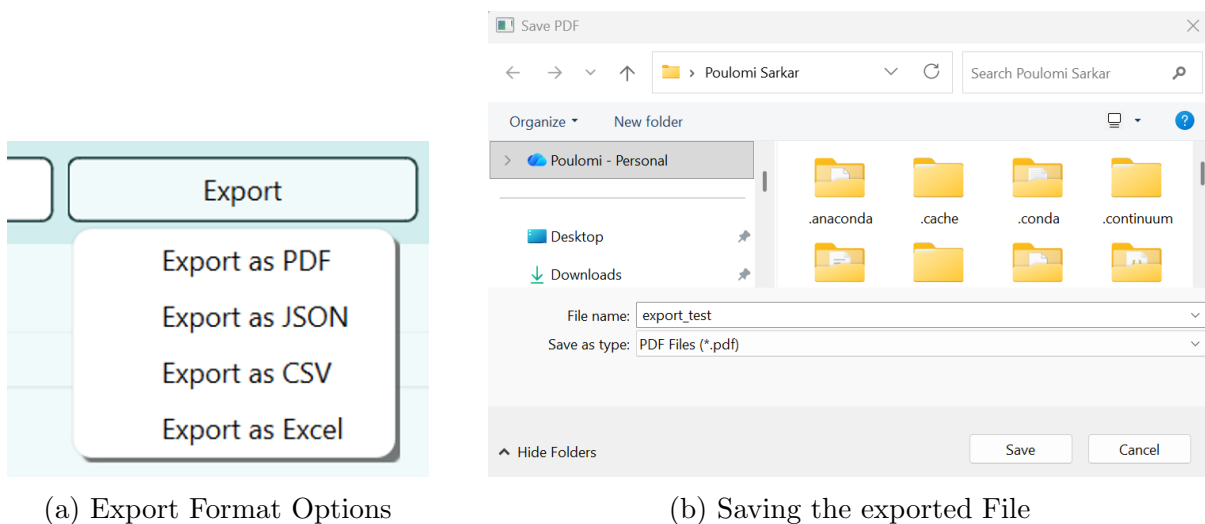


Figure 14: Export

Clicking on this allows the user to export the scanned results. It gives the user 3 options to choose from - PDF, JSON, CSV and Excel. After choosing the format, it allows the user to choose a path to where they want to save it with a name of their choice. It also displays a confirmation message after the export is complete.

Folder Hierarchy Export

Hierarchy (Name)	Size	% of Parent	Last Modified	File Count
D:/	10626.94 MB	100%		3422
└─ anaconda_projects	0.03 MB	0.00%	2025-10-06 15:56	1
└─ db	0.03 MB	100.00%	2025-10-06 15:56	1
└─ project_filebrowser.db	0.03 MB	100.00%	2025-10-06 15:56	0
└─ Andrew Ng Notes.pdf	4.24 MB	0.04%	2025-01-28 15:33	0
└─ AndrewNG Code	14.02 MB	0.13%	2025-10-06 15:58	109
└─ .ipynb_checkpoints	6.32 MB	45.07%	2025-10-06 15:58	41
└─ C1_W1_L3_S1_trainingdata-checkpoint.png	0.08 MB	1.33%	2025-03-22 16:35	0
└─ C1_W1_L4_S1_Lecture_GD-checkpoint.png	0.09 MB	1.40%	2025-03-22 20:19	0

Figure 15: Export as PDF example

```
{
  "% of Parent": "100%",
  "Children": [
    {
      "% of Parent": "0.00%",
      "Children": [
        {
          "% of Parent": "100.00%",
          "Children": [
            {
              "% of Parent": "100.00%",
              "File Count": "0",
              "Last Modified": "2025-10-06 15:56",
              "Name": "project_filebrowser.db",
              "Size": "0.03 MB"
            }
          ]
        }
      ]
    }
  ]
}
```

Figure 16: Export as JSON example

	A	B	C	D	E
1	Name	Size	% of Parent	Last Modified	File Count
2	D:/	10567.79 MB	100%		3401
3	anaconda_projects	0.03 MB	0.00%	06-10-2025 15:56	1
4	db	0.03 MB	100.00%	06-10-2025 15:56	1
5	project_filebrowser.db	0.03 MB	100.00%	06-10-2025 15:56	0
6	Andrew Ng Notes.pdf	4.24 MB	0.04%	28-01-2025 15:33	0
7	AndrewNG Code	14.02 MB	0.13%	06-10-2025 15:58	109
8	.ipynb_checkpoints	6.32 MB	45.07%	06-10-2025 15:58	41
9	C1_W1_L3_S1_trainingdata-checkpoint.png	0.08 MB	1.33%	22-03-2025 16:35	0
10	C1_W1_L4_S1_Lecture_GD-checkpoint.png	0.09 MB	1.40%	22-03-2025 20:19	0
11	C1_W1_Lab01_Jupyter_Notebook-checkpoint	0.00 MB	0.08%	22-03-2025 15:09	0
12	C1_W1_Lab03_alpha_too_big-checkpoint.PNG	0.06 MB	0.93%	22-03-2025 20:15	0
13	C1_W1_Lab03_Model_Representation_Soln-c	0.05 MB	0.71%	22-03-2025 16:17	0
14	C1_W1_Lab04_Cost_function_Soln-checkpoint	1.44 MB	22.75%	22-03-2025 22:30	0
15	C1_W1_Lab05_Gradient_Descent_Soln-check	0.32 MB	5.03%	22-03-2025 20:22	0
16	C1_W2_Lab01_Python_Numpy_Vectorization	0.03 MB	0.50%	24-03-2025 01:08	0

Figure 17: Export as CSV/Excel Example

2.8 Graphical View On/Off

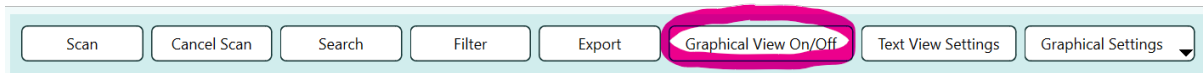


Figure 18: Toggle On/Off for Graphical View

Clicking on this the first time turns on the Graphical View (Treemap) on the right side of the screen using a splitter. The size readjusts based on where the user drags the splitter. Clicking on this button again turns the graphical view off. Hence this button essentially is a toggle.

2.9 Text View Settings

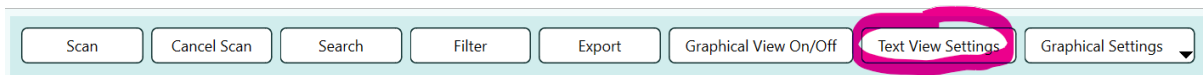
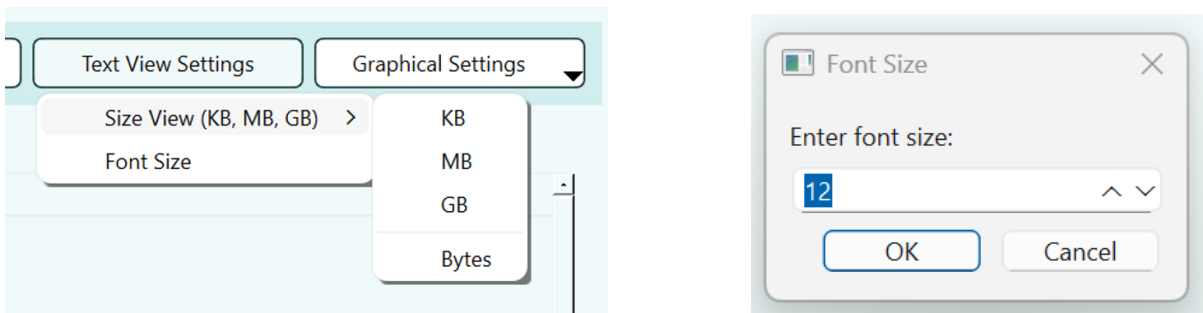


Figure 19: Text View Settings Button



(a) Options under Text View Settings

(b) Font Size Set

Figure 20: Text View Settings

Clicking on this gives the user two options namely - Size View and Font Size. Clicking on Size View allows the user to choose the format of their choice (Bytes, KB, MB or GB) in which they want the file and folder sizes in the main window (text view) to be displayed. Font Size allows the user to choose a font size of their choice for the text view ranging from 8-48.

2.10 Graphical Settings

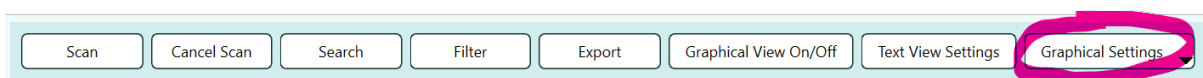


Figure 21: Graphical View Settings Button

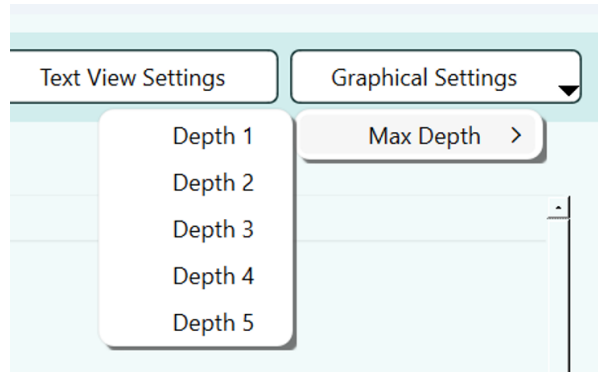


Figure 22: Depth Options

Clicking on this allows the user to choose how many levels they want to see in the graphical view representation (Treemap) of the scanned directory. Depth 1 indicates showing only 1 level subdirectories of the scanned directory and Depth 5 indicating till 5 levels below the original scanned directory.

2.11 Re Scan



Figure 23: Re Scan Button

Clicking on this simply rescans the currently scanned directory again. This might be useful after any delete/rename operations. For delete and rename a rescan is done automatically. However, if the user locally on their system moves/deletes/renames a file or folder then they might want to rescan the directory again to reflect those changes. It saves time, as the user would not have to go on "Scan" and choose a directory from their file explorer to scan again.

2.12 DiskOver

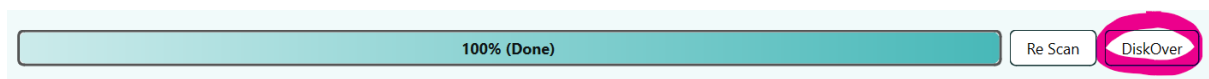


Figure 24: DiskOver Button

This option is a heuristic for trying to clear storage on the device. Clicking on this option gives the user 3 further options to choose from namely - Find Potential Duplicates, Smart Deletion Strategies and View Storage by File Type. The first option suggests possible duplicate files to the user. The second option suggests files to delete by using a scoring index based on size, age, duplicates and type. The third option gives a pie chart view of storage occupied based on format of the files which might allow the user to see what is occupying the most space, whether it is images, videos or something else.

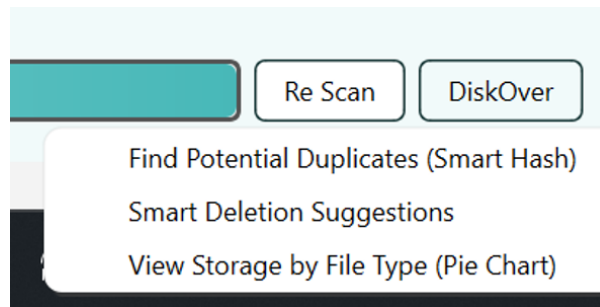


Figure 25: Options under DiskOver

2.13 Progress Bar

The progress bar visually indicates the progress of the current scan. It shows the time elapsed since the scan and shows percentage completed. Before a scan it is at 0% and after the scan is complete it is at 100%.



Figure 26: Mid Scan Example



Figure 27: After the scan is complete

2.14 Full Path

The option to view the full path of a file/folder is available to the user by selecting the option from the context menu by right-clicking on the file/folder that they want to view the path of. After viewing, the user can also copy the path to their clipboard, and can paste it where they want to later. The confirmation messages for the same are also displayed.

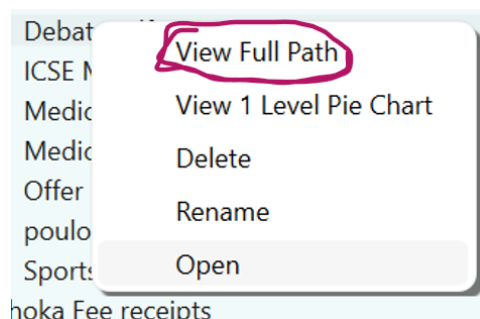
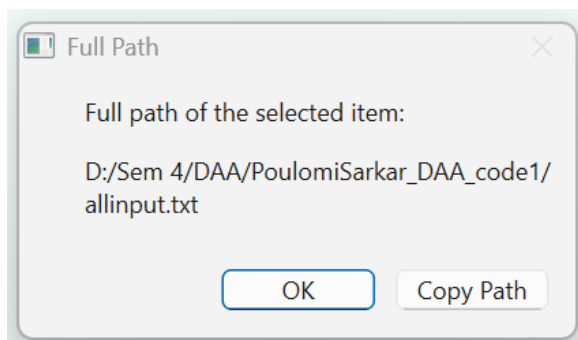
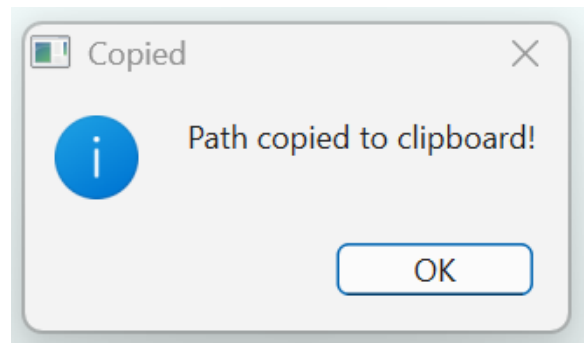


Figure 28: View Full Path Option in Context Menu



(a) View the Full Path Example



(b) Copied to Clipboard

Figure 29: View Full Path Option

2.15 Delete

The option to delete is available to the user by selecting the Delete option from the context menu, by right-clicking on the file/folder that they want to delete. On clicking on Delete, the user will be displayed a message asking to confirm before they want to proceed with the deletion, and after deletion the user will also get a confirmation message for a successful delete operation. This delete option is not a permanent delete. Instead it sends the deleted file/folder to the recycle bin, from where the user may recover it later. For files that are in use and cannot be deleted, a warning message is also displayed to the user. After a successful deletion, an automatic scan of the directory is done, to account for the changes.

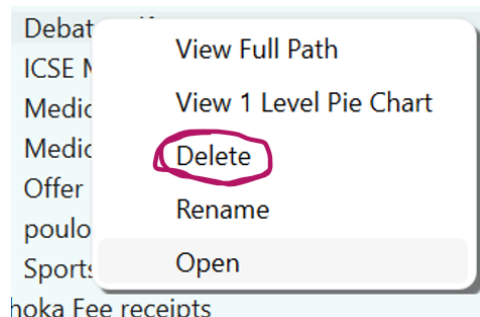


Figure 30: Delete Option in the Context Menu

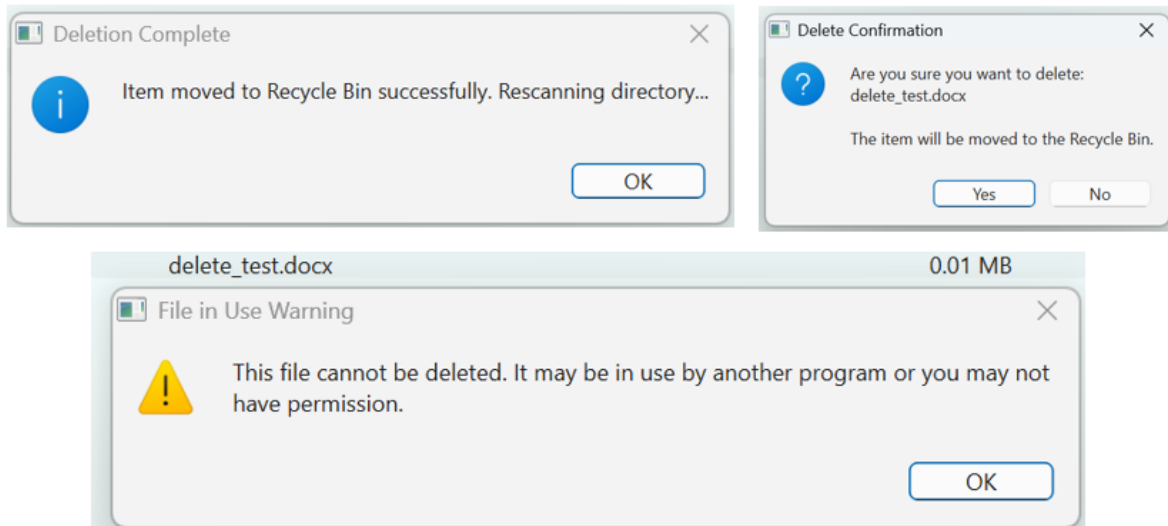


Figure 31: Deletion

2.16 Rename

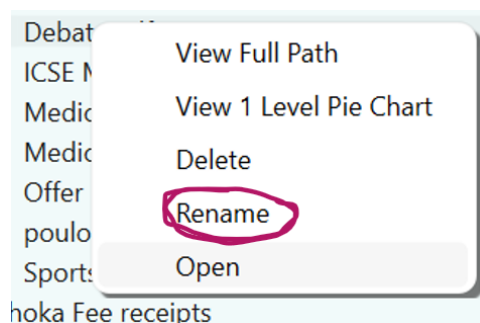


Figure 32: Rename Option in Context Menu

The option to rename is available to the user by selecting the Rename option from the context menu, by right-clicking on the file/folder that they want to rename. The user will then get an option to rename the file/folder to a name of their choice. In the event that a file/folder by that new name already exists in the scanned directory, it shows a warning that the name clashes. After a successful renaming operation, a rescan is done automatically.

2.17 Open File/Folder

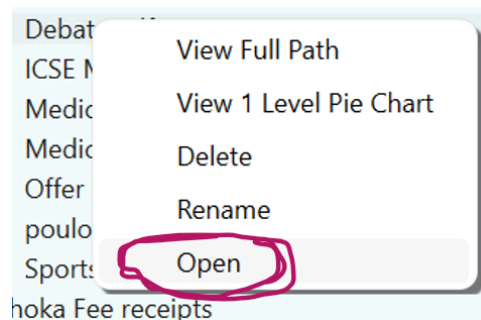


Figure 33: Open Option in the context Menu

The user can open a file/folder by right clicking on it in the text view. Opening a folder, opens it on file explorer. All other files get opened directly using a suitable application. For example: code (VS code), documents (Word).

2.18 1 Level Pie Chart

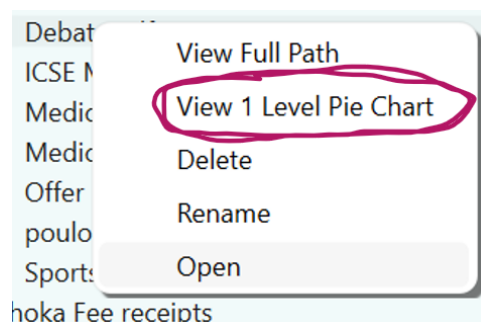


Figure 34: 1 Level Pie Chart Option in Context Menu

The option to view a 1-Level Pie Chart is available to the user by selecting the option from the context menu, by right-clicking on the file/folder that they want to view the Pie Chart for. This option is only available for folders and not files. Incase the user tries this option out for a file, it throws an error.

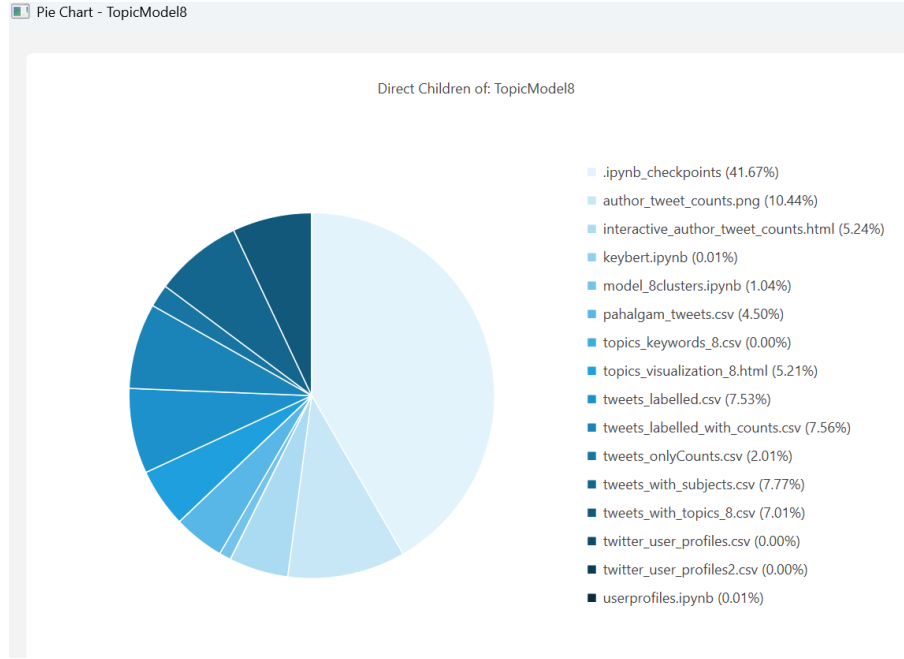


Figure 35: Sample Example of 1-Level Pie Chart

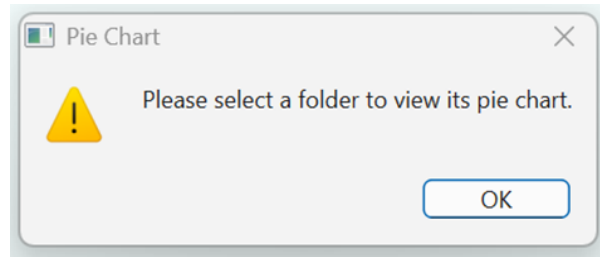


Figure 36: Warning for trying for a File

3 Implementation

We chose **C++** as the core programming language because it provides high performance, efficient memory handling, and direct control over system resources which are all essential for managing large directories and handling file operations smoothly. Its object-oriented structure also made it easier to organize the program into modules for scanning, visualization, and interaction. To build the interface, we used the **Qt framework**(Version 6.9.3), which is a cross-platform toolkit for developing graphical and interactive applications. Qt extends C++ with features that simplify GUI development, offering ready-to-use widgets, layouts, and tools for event handling. It also uses a signal and slot mechanism, which made it easy to link backend logic written in C++ with user actions in the interface. The project was developed in **Qt Creator** (17.0.2 Community Version), an IDE that provides a visual .ui designer for drag-and-drop interface creation, along with built-in build, run, and debugging options that streamlined development. Another key reason for choosing Qt was its cross-platform support, allowing the same code to run on both Windows and macOS with minimal changes. Together, C++ and Qt offered a balance of performance, flexibility, and usability that suited the project perfectly.

Explaining Buttons, Signals and Slots:

In Qt Creator, signals and slots form the core of its event-driven architecture, allowing seamless communication between objects. A signal is emitted when a specific event occurs (for example, when a button is clicked), and a slot is a function designed to respond to that signal. Connections between signals and slots can be made visually using Qt Designer or programmatically using the `connect()` function. For instance, when a user clicks a button in the UI, the button's `clicked()` signal is emitted, which can be connected to a custom slot such as `onScanClicked()` or `exportAsPDF()` in the `MainWindow` class. This mechanism ensures loose coupling: the button does not need to know what the slot does, only that it will be triggered when the signal occurs which makes the application modular, responsive, and easier to maintain.

3.1 Scanning

3.1.1 Requirements and how we analysed it

The requirement as mentioned was to “Perform recursive traversal of a directory tree”. For this purpose we thought it would be useful to have a “Scan” button that would allow the user to choose the directory that they want to do the recursive traversal of. Clicking on this would display the File Explorer of the device, and allow the user to choose what they want to scan. We also added a “Cancel Scan” option to cancel a scan midway or post a scan. It resets the view to no directory currently scanned. We also added a “Re Scan” option to re scan the currently scanned directory. We thought this operation might be useful since sometimes the user makes changes locally. So for example if the user has moved/renamed/deleted a file on their system, and not from treesize, the user can simply hit the rescan option to account for those changes, instead of going to the Scan option and choosing the directory to scan again. Additionally we also do a rescan automatically after a delete or rename operation is done through treesize.

3.1.2 Code Explanation

(i) Scan

- (a) **Overview/Desc-** The Scan operation is responsible for initiating the directory analysis process, reading the file system hierarchy starting from a user-selected root folder, calculating the size and count of files and subdirectories recursively, and populating this data visually into the application's `QTreeWidget` and optionally into the `TreeMapWidget` for graphical visualization.

From the user's perspective, this operation starts when they click the Scan button. The frontend triggers the `onScanClicked()` slot, which acts as a bridge between the UI (frontend) and the logic (backend).

The backend then performs several steps:

- Prompts the user for a directory via a `QFileDialog` allowing the user to choose a directory. Once selected, the application initializes its internal counters, resets any previous results, and begins recursively scanning through the directory hierarchy.
- Initializes variables and clears any old scan results.

- Computes the total number of items to scan for progress estimation.
- Recursively traverses the directory tree using a depth-first search (DFS) implemented in `scanDirectory()`.
- The scan collects the following for each file and folder: The name of the file/folder, Its absolute path on disk, Whether it is a file or a directory, The file size (for files) or the aggregate size of all contained files (for directories), The last modification time, The total file count in each directory
- All of this information is stored in a hierarchical tree structure (`FileNode`), which mirrors the actual directory hierarchy. Once the scan completes, the results are visualized in a `QTreeWidget`, where each node corresponds to a folder or file and displays attributes such as size, percentage of parent folder, and last modification date.
- Updates the `QProgressBar` dynamically as scanning proceeds.
- Builds the hierarchical tree structure of results (`FileNode` objects).
- Populates the GUI tree (`QTreeWidget`) with this data.
- Updates the optional `TreeMap` view for a graphical summary.
- During scanning, the UI remains responsive using `QApplication::processEvents()`, and the progress bar displays real-time progress and elapsed time.

(b) **Key data structures, libraries used & class diagrams**

Qt Classes and Libraries

- `QDir`- Represents a directory in the local file system and provides methods for directory traversal. Used here to list entries (files and subdirectories) within a given path. The method `entryInfoList()` returns a list of `QFileInfo` objects representing each entry in that directory. Filters such as `QDir::NoDotAndDotDot` — `QDir::AllEntries` ensure that only valid files and folders are listed, excluding `."` and `.."` pseudo-directories.
- `QFileInfo`- Encapsulates information about a single file or directory. Provides details like: `fileName()` that returns the item name. `filePath()` that returns its absolute path. `isDir()` / `isFile()` that checks the type. `size()` that gives file size in bytes. `lastModified()` that gives last modification date/time. The combination of `QDir` and `QFileInfo` allows safe and efficient traversal of the filesystem.
- `QTreeWidget`- A hierarchical tree-view widget used to visually display the scanned results. Each directory and file is represented as a `QTreeWidgetItem`. Columns show multiple attributes: name, size, percentage of parent, last modified, and file count.
- `QProgressBar`- Used to display the progress of scanning in percentage terms. Updated dynamically during the recursive traversal to reflect how many items have been processed.
- `QFileDialog`- Provides a standard file/folder picker dialog. The method `getExistingDirectory()` allows the user to select the folder to be scanned.
- `QElapsedTimer`- Used to measure the time elapsed since the start of the scan. The elapsed time is displayed alongside the percentage on the progress bar.

- `QApplication::processEvents()`- Called within the scan loop to process any pending UI events. This prevents the application window from freezing during long scans, maintaining responsiveness.

Custom Data Structure: `FileNode` (`filenode.h`)

The `FileNode` is a custom-defined data structure that represents one node (a file or a folder) in the directory hierarchy. It forms the bridge between the backend computation and the frontend visualization.

```
struct FileNode {
    QString name;
    QString path;
    quint64 size = 0;
    bool isFolder = false;
    quint64 fileCount = 0;
    QList<FileNode> children;
    QDateTime lastModified;
};
```

Class Interactions: The `MainWindow` class acts as the central controller connecting:

- Frontend Widgets (`QTreeWidget`, `QProgressBar`, `QLabel`, `TreeMapWidget`)
- Backend Functions (`scanDirectory()`, `populateTree()`, `countTotalItems()`)
- User Actions (button clicks via `connect()`)

`MainWindow`

`ui` (auto-generated UI components)

`treeMapWidget` : `TreeMapWidget`

`rootNode` : `FileNode`

Functions:

```
onScanClicked()
scanDirectory()
populateTree()
onReScanClicked()
onCancelScanClicked()
```

Data members:

```
cancelScan (bool)
currentDirPath (QString)
totalItems , scannedItems (int)
timer (QElapsedTimer)
```

(c) **Algorithms & heuristics -**

The Scan operation employs a recursive depth-first search (DFS) algorithm to traverse the entire directory structure. Algorithmic Steps:

- Start from the user-selected directory.
- For each item in the directory (retrieved using `QDir::entryInfoList()`):
- If it's a file: Read its size via `QFileInfo::size()`. Increment the current node's size and `fileCount`.

- If it's a directory: Recursively call `scanDirectory()` on that subdirectory. Add the child's total size and file count to the parent node.
- After processing each entry, update: The progress bar (based on number of items processed). The elapsed time using `QElapsedTimer`.
- If `cancelScan` is set to true, the recursion terminates immediately.
- Once the scan completes, the resulting `FileNode` tree is used to populate the GUI view.

Heuristics and Design Considerations:

- DFS over BFS: DFS recursion mirrors the nested structure of directories, making aggregation straightforward and memory-efficient.
- Responsiveness: Instead of using threads, `QApplication::processEvents()` prevents the main window from freezing.
- Pre-counting items: `countTotalItems()` is run before scanning to accurately compute progress percentages.
- Cancel mechanism: The boolean flag `cancelScan` allows an immediate and safe exit from deep recursion.

(d) **Function-wise: short purpose, parameters, output -** `onScanClicked()`

- Purpose: Entry point for initiating a new scan when the Scan button is clicked.
- Parameters: None.
- Opens a folder selection dialog for the user.
- If the user cancels, exits immediately.
- Resets all state variables and progress indicators.
- Stores the selected directory in `currentDirPath`.
- Calls `countTotalItems()` to estimate the number of entries for progress tracking.
- Initializes a new `rootNode` to represent the selected directory.
- Calls `scanDirectory(dirPath, rootNode)` to begin recursive traversal.
- Once completed, populates the tree view via `populateTree()`.
- Updates the graphical `TreeMap` view if enabled.
- Marks the progress bar as 100

`scanDirectory(const QString &path, FileNode &node)`

- Purpose: Core recursive logic that scans a directory and gathers file/folder details.
- Parameters: `path`: The current folder path. `node`: A reference to a `FileNode` object representing this folder.
- Checks if `cancelScan` is true — if yes, stops immediately.
- Creates a `QDir` object with the provided path.
- Retrieves a list of all entries (`QFileInfoList` entries).
- Iterates through each `QFileInfo` entry:
- Creates a `FileNode` for that entry.

- Fills in name, path, size, last modified time, and type (file/folder).
- If it's a folder, recursively calls `scanDirectory()` again.
- Updates cumulative size and file count for the current node.
- Adds this child node to `node.children`.
- Updates progress metrics (percentage and elapsed time).
- Calls `QApplication::processEvents()` to handle UI events.
- Output: Fully populated `FileNode` tree structure with complete size and count data.

`populateTree(const FileNode &node, QTreeWidgetItem *parentItem)`

- Purpose: Converts the backend tree structure (`FileNode`) into a visible tree (`QTreeWidgetItem`) for the user.
- Parameters: `node`: The data node. `parentItem`: Corresponding tree item in the UI.
- Iterates through `node.children`. Creates a `QTreeWidgetItem` for each child.
- Sets column text for: Name (`child.name`), Size (human-readable format), Percentage of parent, Last modified date, File count, Recursively calls itself for child folders.
- Output: Hierarchically populated `QTreeWidgetItem` structure on the UI.

(ii) Re Scan

- Overview/Desc-** The Re-Scan operation allows the user to reanalyze the previously scanned directory without reselecting it. This is particularly useful if files have been added, deleted, or modified since the last scan.
When the Re-Scan button is clicked, the application calls the `onReScanClicked()` function, which uses the stored `currentDirPath` from the previous scan and re-executes the scanning process using the same recursive DFS logic.
- Key data structures, libraries used & class diagrams** Re-Scan uses the same backend classes and UI components as the initial Scan. The only difference is that `currentDirPath` is reused, and no `QFileDialog` is invoked.
The `FileNode` structure and `QDir/QFileInfo` classes again handle the data retrieval and organization.
- Algorithms & heuristics** - Performs a fresh DFS traversal to ensure updated results. Clears the existing tree view and progress bar before re-running. If no previous scan exists (`currentDirPath` is empty), the user is informed via a message box
- Function-wise: short purpose, parameters, output** -
`onReScanClicked()`
 - Purpose: Re-run the directory scan for the same path as the last successful scan.
 - Parameters: None.
 - Checks if a previous scan exists (`currentDirPath` not empty).
 - Resets counters, progress, and clears the current tree.

- Calls `countTotalItems(currentDirPath)` to recompute total entries.
- Initializes a fresh `FileNode` structure.
- mCalls `scanDirectory(currentDirPath, rootNode)` for traversal.
- If not cancelled, populates the `QTreeWidget` with results.
- Updates the progress bar to 100% upon completion.

(iii) Cancel Scan

- (a) **Overview/Desc-** The Cancel Scan operation provides a mechanism to stop an ongoing scan midway. Long recursive scans can take considerable time for large directories, so a safe cancellation feature is critical.

Clicking the Cancel button triggers the `onCancelScanClicked()` slot. This sets a boolean flag `cancelScan = true`, which is periodically checked within `scanDirectory()`. Once the function detects this flag, it stops further traversal and exits recursion safely.

After cancellation, the UI is reset. The progress bar is cleared, tree view emptied, and an informational message displayed.

- (b) **Key data structures, libraries used & class diagrams**

`cancelScan (bool)`: Shared flag between frontend and backend that acts as an interrupt signal. The UI Components are Reset: `QTreeWidget`: Cleared of any partial results. `QProgressBar`: Reset to 0% and cleared. `QMessageBox`: Displays cancellation confirmation to the user.

- (c) **Algorithms & heuristics -**

The algorithm relies on a simple flag-based interrupt mechanism:

```
if (cancelScan) return;
```

This is checked both at the beginning of `scanDirectory()` and inside the loop for every directory entry. This approach avoids multithreading complexity and ensures that recursion terminates cleanly at the next check point. After cancellation, UI state is cleaned up and restored to neutral defaults.

- (d) **Function-wise: short purpose, parameters, output -**
`onCancelScanClicked()`

- Purpose: Allows the user to abort an ongoing scan operation gracefully.
- Parameters: None.
- Verifies that a scan is currently active (`!cancelScan`).
- Sets `cancelScan = true`, signaling all recursive calls to halt.
- Clears all visible scan results (`treeWidget-clear()`).
- Resets progress metrics and counters.
- Displays a message box confirming that the scan has been cancelled.

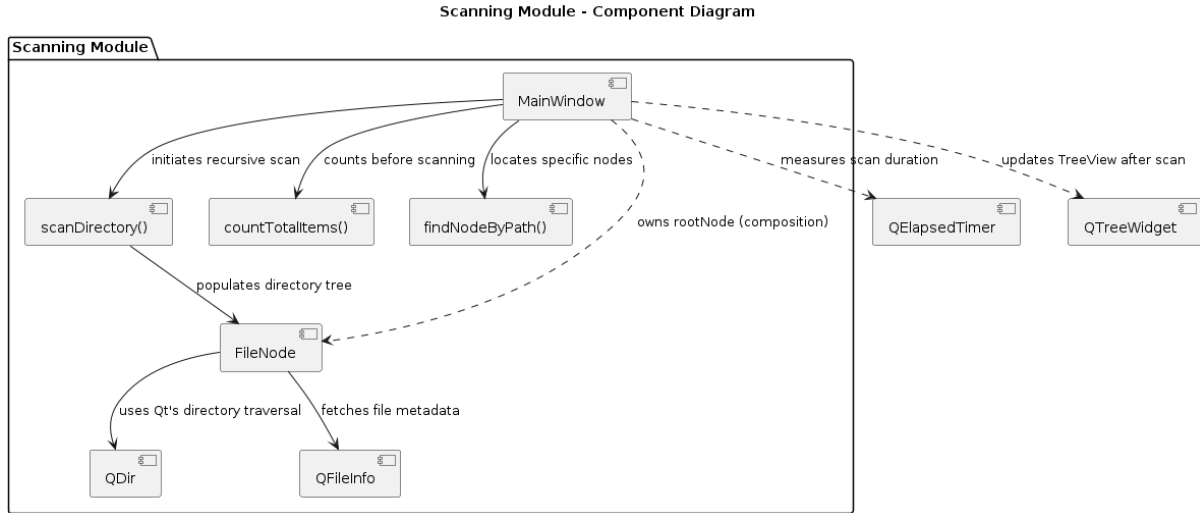


Figure 37: Component Diagram for Scan

3.2 Text and Context Menu

3.2.1 Requirements and how we analyzed it

The requirements as mentioned were to implement "Display total size of folders and files in human-readable format (KB, MB, GB), Display number of files per folder, Provide sorting options by size, name, or file count." To implement this, we thought of having a text view with Name (Name of the File/Folder), Size (In the format chosen by the user - KB, MB or GB), Percentage of Parent (What percentage of the parent folder it takes up), File count (How many files is contained in the folder). The percentage of the parent we thought would be a good addition, as it shows what takes up the most space of the parent folder. Clicking on these headers would also sort in ascending/descending order.

3.2.2 Code Explanation

(i) Text View Columns and Sorting

- (a) **Overview/Desc-** The Tree View representation in the TreeSize application forms the core visual interface through which users can explore the structure and storage distribution of a directory. It is implemented using the `QTreeWidget` class in Qt, which provides a hierarchical tree-based view where each node represents either a folder or a file. Each directory scanned by the program is displayed as a parent node, with its subdirectories and files appearing as child nodes. The tree view features five columns, each serving a specific purpose: Name: Displays the folder or file name. Size: Shows the total size (in bytes) of that item, either directly from the file or computed recursively for folders. % of Parent: Indicates what percentage of the parent folder's total size this item occupies. Last Modified: Displays the last modification timestamp (retrieved from file metadata). File Count: For folders, shows how many files (including subfiles) are contained within; for files, it is set to 1. The columns are initialized and labeled in the constructor of the `MainWindow` class using:

```

ui->treeWidget->setColumnCount(5);
QStringList headers = {"Name", "Size",
"% of Parent", "Last Modified", "File Count"};
ui->treeWidget->setHeaderLabels(headers);

```

Sorting functionality is provided by Qt's built-in sorting mechanism:

```

ui->treeWidget->setSortingEnabled(true);
ui->treeWidget->sortByColumn(0, Qt::AscendingOrder);

```

This allows users to click on any column header to automatically sort items in ascending or descending order based on that column's values. Visually, this part of the interface forms the text-based hierarchical representation of the scanned directory, complementing the graphical TreeMap view. The backend continuously updates this widget as new data is scanned, ensuring the display remains responsive and accurate even during large scans.

(b) Key data structures, libraries used & class diagrams

Qt Widgets and Classes:

- **QTreeWidget**: The main UI component that represents the directory tree. It handles displaying hierarchical data, managing parent-child relationships, and supports sorting, selection, and expansion.
- **QTreeWidgetItem**: Represents each node (file or folder) in the tree. Each item contains multiple columns of data and can have child items to represent subdirectories.
- **FileNode** (Custom Structure): The underlying data model for the tree representation. Each **FileNode** corresponds to one **QTreeWidgetItem** and stores: **name**: **QString**: file/folder name. **path**: **QString**: full system path. **isFolder**: **bool**: true if it's a directory. **size**: **qint64**: cumulative size of the folder or file. **fileCount**: **int**: number of files contained. **lastModified**: **QDateTime**: last modification time. **children**: **QList<FileNode>**: list of sub-items
- **QFileInfo** & **QDir**: Used in the scanning phase to retrieve file metadata (size, modification date, and type). These classes interact closely with the OS to access directory listings and file properties.

Frontend-Backend Interaction:

- The backend scanning logic (using **QDir**, **QFileInfo**, and recursive traversal) builds a tree of **FileNode** objects.
- Once the scan completes, the frontend populates **QTreeWidget** using these nodes through the **populateTree()** function.
- Each column in the tree is filled with formatted data (e.g., percentage strings, formatted timestamps).
- Sorting is automatically handled by the **QTreeWidget** framework whenever a column header is clicked.

This structure ensures a clean separation of the data model (**FileNode**) stores logical information, the UI (**QTreeWidget**) renders it dynamically and the controller (**MainWindow**) links user actions to updates and display refreshes.

(c) **Algorithms & heuristics -**

The data display and sorting logic primarily leverage Qt's built-in tree-sorting algorithm and a recursive population mechanism.

Tree Population Algorithm:

- Implemented via:
`populateTree(const FileNode &node, QTreeWidgetItem *parentItem).`
- Traverses each `FileNode` recursively, creating a new `QTreeWidgetItem` for every file/folder.
- For each child: Name, size, percentage, modification date, and file count are filled into columns.
- If the item is a directory, the function calls itself recursively to fill sub-items.

Percentage of Parent Calculation:

- Percentage is calculated as:

$$\text{percentage} = \frac{\text{child.size}}{\text{parent.size}} \times 100$$

- This gives users a relative sense of how much space each item consumes.

Column Sorting:

- Sorting is automatically handled by `QTreeWidgetItem::sortByColumn()`.
- When a user clicks on a column header, Qt emits internal sorting signals and compares `QTreeWidgetItem` values column-wise.
- For numeric columns (like size and file count), the sorting is numerical; for textual (like names), it's lexicographical.

Dynamic UI Refresh:

- The function `updateTreeDisplay()` refreshes the displayed sizes and percentage values.
- `QApplication::processEvents()` is used during scanning to keep the GUI responsive and allow partial updates.

(d) **Function-wise: short purpose, parameters, output -**

`populateTree(const FileNode &node, QTreeWidgetItem parentItem)`

- Purpose: Recursively builds and fills the tree view from the logical file structure.
- Parameters: `node` current directory or file's `FileNode` and `parentItem` the parent item in `QTreeWidgetItem` where children will be appended.
- Output: Dynamically creates new `QTreeWidgetItem`s under the parent node, representing the full directory hierarchy visually.

`updateTreeDisplay()`

- Purpose: Refreshes all size columns and recalculates percentages for display after the scan or a rescan.
- Parameters: None.
- Output: Updated tree view reflecting current file statistics.

`setSortingEnabled(true)` and `sortByColumn()`

- Purpose: Enables user interaction for column-based sorting in the tree.
- Parameters: Column index (0–4) and Sorting order (ascending or descending).
- Output: Automatically rearranges `QTreeWidgetItem`s based on selected column values.

(ii) Delete (Context Menu)

- (a) **Overview/Desc-** In order to delete the user needs to right click, to get the context menu and then choose the option to delete. Instead of doing a permanent delete, the deleted file goes to the recycle bin. The **Delete** operation allows users to remove files or folders directly from the TreeView interface. When triggered from the right-click context menu, it ensures that the item is safely moved to the Recycle Bin (or Trash) instead of being permanently deleted, providing a safety mechanism against accidental loss. The deletion process is platform-aware, handling file removal differently across Windows, macOS, and Linux, while maintaining consistent user experience. When a user selects “Delete,” a confirmation dialog appears. Upon confirmation, a progress bar updates the user with the status of the deletion, and the UI automatically rescans the directory to refresh the view once the operation completes.

(b) **Key data structures, libraries used & class diagrams**

Qt Widgets and Classes:

- `QTreeWidgetItem`: Represents the selected file/folder to delete.
- `QMessageBox`: Used for confirmation and status messages.
- `QProgressBar`: Displays deletion progress in the UI.
- `QDir`, `QFileInfo`, `QFile`: Used for file system access and property checks.
- `QProcess`: Executes OS-level commands for moving items to Trash on macOS/Linux.

Windows API:

- `SHFileOperationW` via `SHFILEOPSTRUCTW` is used to move files to the Recycle Bin safely using system flags like `FOF_ALLOWUNDO`.

Class Interaction:

- `MainWindow::deleteItem()` handles UI logic, progress updates, and user feedback.
- `MainWindow::moveToRecycleBin()` encapsulates platform-specific deletion logic.
- After success, `onReScanClicked()` refreshes the TreeView to reflect changes.

(c) **Algorithms & heuristics -**

Confirmation and UI Preparation:

- Prompt user with a confirmation dialog.

- **Deletion Execution:** Call `moveToRecycleBin(path)`. Platform-specific handling: Windows: Uses `SHFileOperationW()` with `FOF_ALLOWUNDO` to send to Recycle Bin. macOS: Runs an AppleScript command via `osascript` to delete file through Finder. Linux: Uses `gio trash` or `trash-put` for GNOME/KDE environments.
- **Error Handling:** If deletion fails (e.g., file in use or permission denied), show a warning message.

(d) **Function-wise: short purpose, parameters, output -**

`*deleteItem(QTreeWidgetItem item)`

- **Purpose:** Initiates file deletion, confirms user intent, shows progress, and calls deletion routine.
- **Parameters:** `item`: Tree item selected for deletion.
- **Output:** Updates the UI, shows dialogs, and triggers rescan after successful deletion.

`moveToRecycleBin(const QString &path)`

- **Purpose:** Encapsulates OS-specific logic for moving files/folders to Trash or Recycle Bin.
- **Parameters:** File/folder path.
- **Output:** Returns `true` if deletion succeeded, `false` otherwise.

(iii) **Rename (Context Menu)**

- (a) **Overview/Desc-** The Rename feature allows users to change the name of any selected file or directory directly through the TreeView. When “Rename” is selected from the context menu, an input dialog appears prompting for a new name. The operation updates both the file system and the application’s display by rescanning the directory.

This feature ensures name uniqueness within the same directory and prevents overwriting existing files.

(b) **Key data structures, libraries used & class diagrams**

Qt Widgets and Classes:

- `QInputDialog`: Used to prompt the user for a new file/folder name.
- `QFile`, `QFileInfo`: Used to perform rename and verify existing files.
- `QMessageBox`: For displaying error or success messages.

Frontend–Backend Coordination:

- The user interacts with the `QTreeWidgetItem`.
- The backend uses `QFile` to apply the rename at the filesystem level.
- A rescan ensures the UI reflects the renamed entity accurately.

(c) **Algorithms & heuristics -**

- Prompt the user for a new name via `QInputDialog`.

- Construct the new full path using the old parent directory and new file-name.
- Verify that no file or folder with the same name exists using `QFile::exists()`.
- Attempt to rename using `QFile::rename(oldPath, newPath)`.
- Handle any errors (permissions, in-use files).
- If successful, notify the user and trigger directory rescan.

(d) **Function-wise: short purpose, parameters, output -**

`*renameItem(QTreeWidgetItem item)`

- Purpose: Renames selected file/folder and updates the view.
- Parameters: `item`: Tree item selected for renaming.
- Output: Displays success/failure dialogs; triggers `onReScanClicked()` to update `TreeView`.

(iv) Open (Context Menu)

- (a) **Overview/Desc-** The Open feature allows users to directly open a selected file or folder from the `TreeView` using the system's default application. It appears as an option in the right-click context menu and respects file permissions to prevent unauthorized access.

(b) **Key data structures, libraries used & class diagrams**

Qt Widgets and Classes:

- `QDesktopServices`: Used to open files or folders with the default OS application.
- `QFileInfo`: Checks for file existence and read permissions.
- `QMessageBox`: Displays error messages if access is denied or opening fails.

Frontend-Backend Coordination:

- The user selects an item in the `QTreeWidgetItem`.
- Backend validates permissions and invokes `QDesktopServices::openUrl()` to launch the item.

(c) **Algorithms & heuristics -**

- Retrieve the full file/folder path from the selected item.
- Check existence and readability using `QFileInfo`.
- If valid, call `QDesktopServices::openUrl(QUrl::fromLocalFile(path))`.
- Display warnings for missing or restricted-access items.

(d) **Function-wise: short purpose, parameters, output -**

`*openItem(QTreeWidgetItem *item)`

- Purpose: Opens the selected file/folder via the system's default application.
- Parameters: `item` – Tree item representing the target file/folder.
- Output: Opens the file/folder or displays an error message if access is denied.

(v) 1-Level Pie Chart

- (a) **Overview/Desc-** The 1-Level Pie Chart feature offers a visual representation of how space is distributed among the immediate children of a selected directory. Triggered from the context menu, it opens a resizable dialog displaying a dynamic pie chart using Qt Charts, where each slice represents a child folder or file and its proportion of the parent's total size.

(b) **Key data structures, libraries used & class diagrams**

Qt Classes:

- **QChart, QChartView, QPieSeries, QPieSlice:** Components from the QtCharts module used for plotting the pie chart.
- **QDialog, QVBoxLayout:** Used to host and layout the chart display.

Custom Structures:

- **FileNode:** Provides the size and name of the current folder's children.

Class Interaction:

- **showPieChart()** finds the corresponding **FileNode** using **findNodeByPath()**.
- Child sizes are iterated and added to the **QPieSeries**.
- The chart is rendered within a modal dialog for user inspection.

(c) **Algorithms & heuristics -**

- Retrieve selected folder path and find corresponding **FileNode** using **findNodeByPath()**.
- Validate that the node is a directory and contains children.
- Compute percentage of each child relative to the parent:

$$\text{percentage} = \frac{\text{child.size}}{\text{parent.size}} \times 100$$

- Create **QPieSeries**, append slices for each child, and label them.
- Initialize **QChart** and **QChartView**, set legend alignment, and embed in dialog layout.
- Display chart in a resizable modal dialog.

(d) **Function-wise: short purpose, parameters, output -**

***showPieChart(QTreeWidgetItem item)**

- **Purpose:** Displays a pie chart showing space distribution among direct subitems of a directory.
- **Parameters:** **item:** selected folder node in TreeView.
- **Output:** Modal dialog displaying a visual 1-level chart of the folder's composition.

findNodeByPath(FileNode &node, const QString &path)

- **Purpose:** Searches recursively for the corresponding **FileNode** based on its path.
- **Parameters:** **node:** root or current node; **path:** search target.

- Output: Pointer to the matching `FileNode`, or `nullptr` if not found.

(vi) View Path

- (a) **Overview/Desc-** The **View Path** operation provides a convenient way for users to view and copy the absolute file system path of any selected `TreeView` item. When invoked, a dialog displays the full path and includes a “Copy Path” button that copies the path to the system clipboard.

(b) Key data structures, libraries used & class diagrams

Qt Widgets and Classes:

- `QMessageBox`: Displays the file path and the “Copy Path” button.
- `QClipboard`: Handles copying the path text to the system clipboard.
- `QTreeWidgetItem`: Provides access to the file path stored in `Qt::UserRole`.

Class Interactions:

- `showFullPath()` is connected to the `QAction` in the context menu.
- The function interacts directly with the clipboard and dialog subsystems.

(c) Algorithms & heuristics -

- Retrieve the path from the item using `item->data(0, Qt::UserRole)`.
- Display it using a `QMessageBox` with an added “Copy Path” action button.
- If the user clicks “Copy Path,” copy it using `QClipboard::setText()`.
- Show a confirmation message “Path copied to clipboard!”

(d) Function-wise: short purpose, parameters, output -

*`showFullPath(QTreeWidgetItem item)`

- Purpose: Shows full path of the selected file/folder and allows copying it to clipboard.
- Parameters: `item` selected tree item.
- Output: Modal dialog with the path and optional copy confirmation.

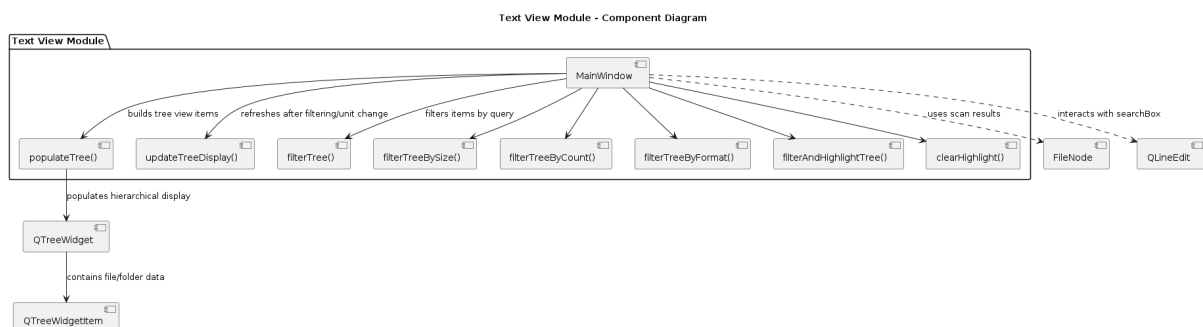


Figure 38: Component Diagram for Text View

3.3 Tool Bar and Progress Bar

3.3.1 Requirements and how we analysed it

The requirements that were asked for were: Search Functionality: Allow quick search for files/folders by name. Progress Indicators: Show progress bar during scans. Allow exporting reports to CSV, JSON, or PDF. Display total size of folders and files in human-readable format (KB, MB, GB). We thought of adding a Filter option so the user can filter by count, size, type, format and to reset the filter. For the search matches, the files/folders collapse to the match and get highlighted, similarly for filter they collapse and show in bold. We also added functionalities to change font size. For the Export option, they are displayed as a list of all the files which are in the directory along with their sizes, percentage of parent values, file counts and last modified dates without showing the hierarchy.

3.3.2 Code Explanation

(i) Search

- (a) **Overview/Desc-** Search allows the user to locate files or folders by name in the tree view. It provides real-time filtering and highlights matching items while preserving tree hierarchy visibility. Empty queries restore the original view.
- (b) **Key data structures, libraries used & class diagrams-**
 - **QLineEdit:** Used for the search input box.
 - **QTreeWidgetItem:** Represents files/folders in a hierarchical tree.
 - **QBrush/QColor:** To highlight matches.
 - **QString:** For string comparisons and trimming.
 - **MainWindow:** Contains the UI components and implements the search logic.
 - **Recursive traversal:** Functions walk the tree recursively for filtering/highlighting.
- (c) **Algorithms & heuristics-**
 - Traverse each tree item recursively.
 - Compare item text with search query (case-insensitive).
 - Highlight matching items (light yellow) and bold their font if needed.
 - Collapse non-matching branches and expand branches containing matches.
 - Restore original visibility when the search query is empty.
- (d) **Function-wise: short purpose, parameters, output-**
 - **onSearchClicked():** Toggles visibility of the search box; connects input changes to **onSearchTextChanged()**. No parameters, no output.
 - **onSearchTextChanged(const QString &text):** Filters and highlights the tree based on the input query. Parameter: search string; output: updates tree view.
 - **filterAndHighlightTree(QTreeWidgetItem*, const QString &):** Recursively matches items, highlights them, hides non-matches, expands relevant branches. Returns a boolean indicating if the item should be visible.

- `clearHighlight(QTreeWidgetItem*)`: Clears highlighting from an item and all its children.

(ii) Filter

- Overview/Desc-** Filter allows selective viewing of items based on file size, count, format, or type. Users can also reset filters to restore the full tree view.
- Key data structures, libraries used & class diagrams-**
 - **QMenu & QAction:** For creating filter menus and actions.
 - **QInputDialog:** To get filter criteria from users.
 - **QTreeWidgetItem:** Tree items that are filtered.
 - **QSet/QStringList:** To collect available file formats/types.
 - **Recursive lambda functions:** For tree traversal and filtering.
- Algorithms & heuristics-**
 - Recursively traverse tree items.
 - Determine match using criteria (size comparison, file count, format/type).
 - Highlight matched items and collapse non-matching branches.
 - Reset restores visibility and font styles for all items.
- Function-wise: short purpose, parameters, output-**
 - `onFilterClicked()`: Shows a menu with filter options and connects actions to specific filter functions.
 - `filterBySize()`: Filters files smaller or larger than a user-specified size. Parameters: size value, comparison choice; output: tree updated.
 - `filterByFileCount()`: Filters based on number of files. Parameters: count value, comparison choice; output: tree updated.
 - `filterByFormat()`: Filters items matching a chosen file extension. Parameter: selected extension; output: tree updated.
 - `filterByType()`: Filters items matching file type categories (Images, Videos, etc.). Parameter: selected type; output: tree updated.
 - `resetFilter()`: Restores all tree items to visible, resets fonts and collapses children.

(iii) Export

- Overview/Desc-** The export functionality allows users to save the visualized directory hierarchy and metadata in multiple formats — PDF, JSON, CSV, and Excel — while preserving the hierarchical structure. Upon clicking the “Export” button, a dropdown menu is displayed with options for the desired format. Each format-specific export function recursively traverses the `QTreeWidgetItem` and writes data in a structured and user-friendly way. This enables portability, data sharing, and offline analysis.
- Key data structures, libraries used & class diagrams-** The implementation primarily uses Qt classes for GUI and file operations:
 - **QMenu, QAction** — To create the export dropdown and connect each export option to its corresponding slot function.

- `QFileDialog`, `QFile`, `QTextStream` — For file selection, creation, and writing textual data (CSV, Excel).
- `QJsonObject`, `QJsonArray`, `QJsonDocument` — For serializing the hierarchical data into JSON.
- `QPrinter`, `QTextDocument` — For generating formatted PDF exports from HTML content.
- `QTreeWidgetItem` — To recursively access and extract directory information displayed in the UI.

The core logic resides within the `MainWindow` class, where each export function (`exportAsPDF()`, `exportAsJSON()`, `exportAsCSV()`, `exportAsExcel()`) is implemented as a separate slot.

(c) **Algorithms & heuristics-** The export logic follows a recursive traversal of the directory hierarchy:

- A lambda function (`std::function<void(QTreeWidgetItem*, ...)>`) is used to recursively visit each `QTreeWidgetItem`.
- For CSV and Excel, indentation or prefix characters (e.g., `|-`) visually represent hierarchy levels.
- JSON export builds a tree structure where each node includes metadata and a “Children” array if applicable.
- PDF export uses HTML table generation for formatting and hierarchy indication before rendering to PDF through `QTextDocument`.
- Excel export internally uses a CSV-compatible format with level-based indentation, ensuring compatibility with spreadsheet tools.

(d) **Function-wise: short purpose, parameters, output-**

- `void MainWindow::onExportClicked()`
 - **Purpose:** Displays an export menu with multiple format options and connects actions to export slots.
 - **Parameters:** None.
 - **Output:** Opens a dropdown menu linked to export functions.
- `void MainWindow::exportAsCSV()`
 - **Purpose:** Saves directory hierarchy as a CSV file with each row representing a node.
 - **Parameters:** None.
 - **Output:** Creates a user-selected `.csv` file with hierarchical structure preserved using prefix characters.
- `void MainWindow::exportAsJSON()`
 - **Purpose:** Exports data in structured JSON format for reusability and programmatic access.
 - **Parameters:** None.
 - **Output:** Writes a `.json` file containing hierarchical directory data with nested child arrays.
- `void MainWindow::exportAsPDF()`
 - **Purpose:** Generates a formatted PDF document with an HTML-rendered table representing the hierarchy.

- **Parameters:** None.
- **Output:** Creates a .pdf file containing a readable folder hierarchy with all relevant metadata.
- **void MainWindow::exportAsExcel()**
 - **Purpose:** Produces an Excel-compatible CSV file with indentation-based hierarchy levels and structured metadata.
 - **Parameters:** None.
 - **Output:** Saves an Excel-compatible file (.csv) maintaining directory depth and attributes.

(iv) Changing Size Format (Text View Settings)

- (a) **Overview/Desc-** Allows users to display file sizes in Bytes, KB, MB, or GB in the tree view. Updates all visible items according to the selected unit.
- (b) **Key data structures, libraries used & class diagrams-**
 - **QMenu & QAction:** For unit selection.
 - **QTreeWidgetItem:** Stores raw size in UserRole for conversions.
 - **Lambda functions:** For recursive updates.
- (c) **Algorithms & heuristics-**
 - Store raw size in bytes for all items.
 - Convert to chosen unit using multiplication/division.
 - Update displayed text with 2 decimal precision.
 - Recursive traversal ensures all children are updated.
- (d) **Function-wise: short purpose, parameters, output-**
 - **updateTreeDisplay():** Iterates all tree items and converts stored raw sizes to current unit. Parameters: none; output: updated text in tree.
 - Connected QAction triggers in **onTextViewSettingsClicked()** set the **currentUnit** and call **updateTreeDisplay()**.

(v) Change Font Size (Text View Settings)

- (a) **Overview/Desc-** Enables dynamic resizing of the tree view font to improve readability.
- (b) **Key data structures, libraries used & class diagrams-**
 - **QInputDialog:** To input font size.
 - **QFont:** Adjusts tree font.
 - **QTreeWidgetItem:** Updated to new font size.
- (c) **Algorithms & heuristics-**
 - Prompt user for font size within a valid range (8–48).
 - If valid, update the QTreeWidgetItem font globally.
- (d) **Function-wise: short purpose, parameters, output-**
 - **onFontSizeSelected():** Prompts user for integer font size; updates tree widget font. Parameters: none (input via dialog); output: updated tree font.

(vi) Progress Bar

- (a) **Overview/Desc-** Visual feedback to indicate long-running operations (like scanning or exporting large directories). Displays completion percentage dynamically.
- (b) **Key data structures, libraries used & class diagrams-**
 - **QProgressBar:** GUI widget showing progress.
- (c) **Algorithms & heuristics-**
 - Increment progress as items are processed.
 - Update value on the GUI thread to reflect completion percentage.
 - Reset or hide progress bar when operation finishes.
- (d) **Function-wise: short purpose, parameters, output-**
 - **setValue(int):** Updates progress bar.
 - **show()/hide():** Display or hide progress bar at start/end of operation.

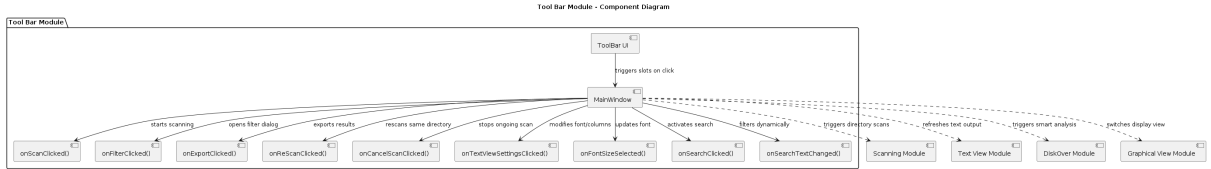


Figure 39: Tool Bar Component Diagram

3.4 Graphical View

3.4.1 Requirements and how we analysed it

From the functional and UI/UX requirements stated for the project, we will be stating and understanding the portions relevant to the graphical view here. Further, we will detail how we have incorporated them along with the assumptions we have made.

1: Support graphical visualization of folder sizes (bar chart, pie chart, or treemap).

- Opted for a treemap visualization over a bar chart/pie chart as it was most intuitive to understand from the user's perspective.
- A treemap visualization allows the user to view several levels deep into their directory while being able to compare it with the what
- To make the visualization as appealing and readable as possible we tested several algorithms for visualizations: slice and dice, squarified, and squarified+.
- We also tested several modifications to the above algorithms to be able to balance clutter and detail. To do this we set several rules on clubbing and naming which will be detailed below.

2: Show metadata when hovering over items.

- To enable this feature we tracked the movement of the mouse across the rectangles that represented the files/folders.
- Details on hovering in the graphical view are name,
- These are accessed from the QTreeWidget after the initial scan and text view are generated. Only those details that can be accessed in this format are displayed.

3.4.2 Code Explanation

(i) TreeMapWidget

(a) **Overview/Desc-** The TreeMapWidget is a custom Qt widget that visualizes the hierarchical file system in a squarified treemap layout. It is designed to complement the standard QTreeWidget by giving a proportional, visual view of directory contents. This module handles:

- Rendering files/folders as rectangles, sized according to % of parent folder occupation
- Recursive layout of children with the squarified algorithm chosen for better aspect ratios.
- Depth-based filtering and clubbing of small children into virtual nodes.
- Mouse interactions including click-to-drill-down, hover tooltips, and up-navigation.
- Integration with the main window (MainWindow) tree view for synchronized navigation.

The widget uses FileNode objects stored in QTreeWidgetItemItems to retrieve metadata like size, path, last modified time, and children.

(b) Key data structures, libraries used & class diagrams-

• FileNode

- Purpose: Represents a file or folder in the file system.
- Attributes:
 - * QString path: Full path of the file/folder.
 - * quint64 size: Size in bytes (for directories, aggregated size of children)
 - * QList<FileNode*> children: List of child FileNodes for folders.
 - * QDateTime lastModified: Last modification timestamp.
- Usage: Stored in Qt::UserRole of QTreeWidgetItemItems for fast access during rendering and interaction.

• TreeMapWidget::ChildDesc

- Purpose: Describes a child node for rendering in the treemap.
- Attributes:
 - * QTreeWidgetItem* item: Pointer to the actual tree item (if not virtual).
 - * QString label: Display label.
 - * quint64 size: Size in bytes.

- * bool isVirtual: True if this child represents a clubbed node.
 - * QVector<QTreeWidgetItem*> clubbedItems: Items merged into this virtual child.
 - Usage: Sorting, depth-clubbing, and recursive treemap calculations. Helps map rectangles to items for hit detection.
 - **TreeMapWidget::RectMapEntry**
 - Purpose: Maps rendered rectangles to their corresponding items.
 - Attributes:
 - * QRectF rect: Position and size of rectangle in widget coordinates.
 - * ChildDesc child: Associated child or virtual node.
 - Usage: Used in mousePressEvent and mouseMoveEvent to detect which rectangle was clicked or hovered.
- (c) **Algorithms & heuristics-** This section documents the specific rules, thresholds, and logic used by the TreeMapWidget to determine visibility, layout, and grouping of items.
- (a) **Node Visibility Filtering**
- Purpose: Exclude nodes whose total size is too small to be visually meaningful relative to the absolute root.
 - Implemented Using: passesRootVisibility(QTreeWidgetItem *item, quint64 absoluteRootSize)
 - Rule: A node is drawn only if $\frac{\text{itemSize}}{\text{absoluteRootSize}} \times 100 \geq \text{m_minRootPercent}$
 - Default Threshold: $\text{m_minRootPercent} = 0.2$ (items smaller than 0.2% of root are hidden)
 - Effect: Applies to all nodes except root; threshold adjustable dynamically via setMinRootPercent(double pct)
- (b) **Depth-Based Child Limiting**
- Purpose: Prevent visual overcrowding by reducing number of child nodes at deeper levels.
 - Implemented Using: allowedChildrenForDepth(int depth) and applyDepthClubbing(std::vector<ChildDesc>&, int)
 - Computation Rule: $\text{allowedChildren} = \max(0, \text{m_maxVisibleRoot} - 2 \times \text{depth})$
 - Parameters: $\text{m_maxVisibleRoot} = 8$
 - Behavior:
 - Depth 0 (root): 8 children, full detail
 - Depth 1: 6 children, slight compression
 - Depth 2: 4 children, moderate compression
 - Depth 3: 2 children, heavy compression
 - Depth 4+: 0 children, fully clubbed
- (c) **Clubbing / “Virtual Node” Aggregation Logic**
- Purpose: Merge excess child nodes into a single virtual item when count exceeds allowed per-depth limit.
 - Implemented Using: applyDepthClubbing(std::vector<ChildDesc>& children, int maxVisible)

- Algorithm:
 - Sort children descending by size
 - If $\text{maxVisible} \leq 0$: all children aggregated into one virtual node
 - If number of children $> \text{maxVisible}$: keep top maxVisible , club rest into virtual node with label " + [N Files/Folders]"

(d) **Squarified Treemap Layout Algorithm**

- Purpose: Assign each node a rectangle proportional to its size while preserving aspect ratios close to 1:1.
- Implemented Using: `squarifyLayout(const std::vector<quint64>& sizes, const QRectF& rect)`
- Key Steps:
 - Normalize item sizes to fractions of total
 - Maintain a working "row" of rectangles
 - Iteratively add items to row, compute worst aspect ratio with `worstRatio(row, side)`
 - If adding next item worsens ratio, commit row (slice horizontally or vertically depending on width vs height)
 - Repeat until all items placed
- Aspect Ratio Metric: $r = \max \left(\frac{\text{side}^2 \times \text{max}(v)}{\text{sum}(v)^2}, \frac{\text{sum}(v)^2}{\text{side}^2 \times \text{min}(v)} \right)$

(e) **Label Suppression & Elision**

- Purpose: Dynamically truncate text to fit within rectangles.
- Rules: At least 3 visible characters before ellipsis; always show folder/file prefix and percentage; suppressed entirely if too small
- Implemented Using: `drawItem()` and `drawVirtualItem()`

(f) **Minimum Rectangle Visibility Safeguards**

- Purpose: Avoid drawing or recursing into rectangles too small
- Condition: Skip if $r.\text{width}() \leq 1.0$ or $r.\text{height}() \leq 1.0$

(g) **Interactive Navigation Heuristics**

- Up button active only when $m_currentRoot \neq \text{nullptr}$
- Rectangle width fixed to 60 px
- Tooltip display only for non-virtual items with: Name, Human-readable size, % of parent, Last modified timestamp

(d) **Function-wise: short purpose, parameters, output-**

- `setSourceTree(QTreeWidget tree, int maxDepth)*`: Sets the source tree for rendering; triggers repaint
- `clearSource()`: Clears the source tree pointer; resets internal mappings
- `setCurrentRoot(QTreeWidgetItem item)`: Changes root; updates percentages and rectangle sizes
- `clearCurrentRoot()`: Resets root to top-level; recalculates layout
- `itemRawSize(QTreeWidgetItem item)*`: Returns byte size
- `humanReadableSize(quint64 bytes)`: Converts bytes to KB/MB/GB string
- `allowedChildrenForDepth(int depth)`: Returns max children to render at given depth

- `passesRootVisibility(QTreeWidgetItem item, quint64 absoluteRootSize)*`: Returns true if item is visible
- `buildFilteredChildren(QTreeWidgetItem parent, quint64 absoluteRootSize)*`: Returns filtered vector of `ChildDesc`
- `applyDepthClubbing(vector<ChildDesc>& children, int maxVisible)`: Converts excess children into virtual `ChildDesc`
- `squarifyLayout(vector<quint64> sizes, QRectF rect)`: Returns rectangles proportional to sizes
- `drawItem(QPainter& painter, QRectF rect, QTreeWidgetItem item, int depth, quint64 absoluteRootSize)*`: Recursively draws children; maps rectangles
- `drawVirtualItem(QPainter& painter, QRectF rect, ChildDesc& virtualChild, QTreeWidgetItem parentItem)*`: Renders clubbed items
- `drawUpButton(QPainter& painter, QRectF rect)`: Draws “Up” button
- `drawLegend(QPainter& painter, QRectF rect)`: Draws depth-based color legend
- `mousePressEvent(QMouseEvent event)*`: Detects clicks; opens menu for virtual nodes
- `mouseMoveEvent(QMouseEvent event)*`: Detects hover; shows tooltips
- `navigateUp()`: Moves root to parent; triggers repaint
- `showClubbedItemsMenu(QVector<QTreeWidgetItem> items, QPoint globalPos)*`: Shows context menu for virtual nodes

3.5 DiskOver

3.5.1 Requirements and how we analysed it

From the functional and UI/UX requirements stated for the project, this section covers the OverDisk / Free Space Analysis portion. The purpose of this module is to provide the user with additional insights and actionable features for optimizing storage space beyond the basic directory scan and graphical visualization.

Functional Goals:

1. Identify potential duplicate files using a fast partial-hash technique.
2. Generate intelligent Smart Deletion Suggestions based on file size, age, and type heuristics.
3. Present a storage composition view as a pie chart, visualizing how different file categories occupy disk space.

Design Rationale:

1. The free space options extend the core scanning functionality, using the existing FileNode hierarchy as input.
2. The design prioritizes speed and usability, leveraging quick partial-hash comparisons instead of full-file hashing.
3. Analytical results (duplicates, deletion candidates, and pie charts) are presented interactively via Qt dialogs for minimal UI disruption.
4. The interface is invoked through a single menu (onFreeSpaceClicked()), giving the user a centralized entry point for space optimization tasks.
5. These features collectively make OverDisk a post-analysis extension — providing a deeper understanding of how storage is utilized and actionable suggestions to reclaim space safely.

3.5.2 Code Explanation

(i) OverDisk

- (a) **Overview/Desc-** The OverDisk module (integrated within MainWindow) is responsible for analyzing existing scan results to detect redundancy, estimate cleanup potential, and visualize space usage by file types.

It does not perform filesystem scanning itself — instead, it operates on the FileNode tree produced by the scan module. The module provides three main functionalities, each invoked via the “Smart Free Space Options” menu:

- Find Potential Duplicates (Smart Hash)
- Smart Deletion Suggestions
- View Storage by File Type (Pie Chart)

Each feature is implemented as an independent function, using supporting structures like SmartFileInfo for normalized scoring and data aggregation.

(b) Key Data Structures & Helper Classes-

• SmartFileInfo

- Purpose: Represents simplified metadata for analytical computations.
- Attributes:
 - * QString name: File name.
 - * QString path: Full file path.
 - * QString extension: File extension (used for categorization).
 - * quint64 size: File size in bytes.
 - * QDateTime lastModified: Timestamp of last modification.
 - * double sizeScore: Normalized score (0–1) for size-based weighting.
 - * double ageScore: Normalized score for file age.
 - * double typeScore: Heuristic weight for low-priority file types (e.g., .tmp, .bak, .log).
 - * double duplicateScore: Placeholder score if duplicate detection finds identical hashes.
 - * double totalScore: Weighted sum of all factors representing deletion priority.
- Usage: SmartFileInfo objects are collected recursively from the FileNode hierarchy and used across all OverDisk analyses.

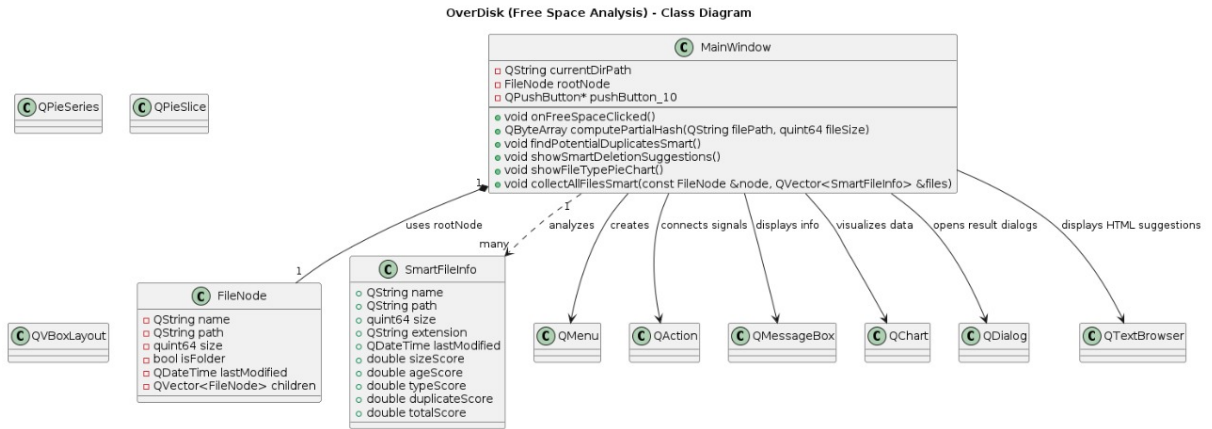


Figure 41: Class Diagram for DiskOver

(c) Algorithms & Heuristics-

(a) Duplicate Detection (Smart Hashing)

- Purpose: Detect likely duplicate files using size-grouping and partial MD5 hashing to improve speed.
- Implemented Using: findPotentialDuplicatesSmart() and computePartialHash(const QString &filePath, quint64 fileSize)
- Key Steps:
 - Collect all files recursively using collectAllFilesSmart().
 - Group files by size (QMap<quint64, QVector<SmartFileInfo>>) to avoid unnecessary hash comparisons.

- For each same-size group:
 - * Compute partial MD5 hash using `computePartialHash()`, which reads only the first and last 1 MB of the file.
 - * Group files with matching hashes together.
- Display likely duplicate sets in a formatted message dialog.
- Optimization:
 - Files smaller than 1 MB are fully read.
 - Partial hash drastically reduces computation for large files without sacrificing much accuracy.
- Outcome: Duplicate candidates are listed as “Group N – X files likely identical” with file paths under each group.

(b) **Smart Deletion Suggestion Algorithm**

- Purpose: Suggest candidate files for deletion based on weighted heuristic scoring.
- Implemented Using: `showSmartDeletionSuggestions()`
- Key Steps:
 - Collect all files with `collectAllFilesSmart()`.
 - Compute normalized scores:
 - * Size Score: proportional to file size vs. maximum size in directory.
 - * Age Score: based on how old the file is (`daysOld / 365`, capped at 1).
 - * Type Score: set to 1.0 if the file extension is in low-priority types (`tmp`, `bak`, `log`, `cache`).
 - * Duplicate Score: currently unused placeholder.
 - Weighted total formula: $\text{totalScore} = 0.4 * \text{sizeScore} + 0.3 * \text{ageScore} + 0.2 * \text{typeScore} + 0.1 * \text{duplicateScore}$
 - Sort files descending by `totalScore`.
 - Display top 10 candidates in an HTML-styled dialog.
- Displayed Data:
 - File name and path
 - Composite score
 - Breakdown of size/age/type factors (in percentage)
 - HTML-based report for readability and ranking visualization.
- Effect: Highlights files that are both large and old, or low-value temporary types, helping users identify redundant data.

(c) **Storage Composition Visualization (Pie Chart)**

- Purpose: Show how storage is distributed by file type categories for the selected directory.
- Implemented Using: `showFileTypePieChart()`
- Key Steps:
 - Collect all files using `collectAllFilesSmart()`.
 - Define category mappings (Images, Videos, Documents, Audio, Archives, Code, Others).

- Aggregate total size per category: `categorySizes[category] += file.size`
- Compute percentage relative to total scanned size.
- Render results using QtCharts (QPieSeries, QChartView).
- Output: A pie chart dialog showing categories and their respective storage percentages, with labeled slices and legend at the bottom.

(d) **Supporting Utility Functions-**

- `onFreeSpaceClicked()`: Entry point for the OverDisk menu; displays options as context menu; connects actions to slots; prevents execution if no directory has been scanned.
- `computePartialHash(const QString &filePath, quint64 fileSize)`: Computes quick MD5 hash from first and last 1 MB of file data.
- `collectAllFilesSmart(const FileNode &node, QVector<SmartFileInfo> &files)`: Recursively traverses the file tree; adds each file as a SmartFileInfo entry; skips folders but continues into subdirectories.

(e) **Function-wise Description-**

- `onFreeSpaceClicked()`: Displays “Smart Free Space Options” menu and connects UI actions.
- `findPotentialDuplicatesSmart()`: Groups files by size, computes partial hashes, and lists likely duplicates.
- `computePartialHash(QString path, quint64 fileSize)`: Reads first and last 1 MB of file and returns MD5 hash.
- `showSmartDeletionSuggestions()`: Scores and ranks files by redundancy potential, displaying top results in HTML format.
- `showFileTypePieChart()`: Aggregates file sizes by type and displays them as a pie chart.
- `collectAllFilesSmart(const FileNode &node, QVector<SmartFileInfo> &files)`: Recursively collects file metadata for all analytical operations.

(f) **Integration and Role-**

- The OverDisk module serves as the analytical and cleanup extension to the scanning system.
- While the Graphical View focuses on spatial hierarchy and proportional visualization, OverDisk provides quantitative analysis and optimization tools: giving users insights into redundancy, category-based distribution, and intelligent cleanup suggestions.
- Its modular design allows future expansion (e.g., full duplicate deletion, combined scoring with hash data, and exportable reports), maintaining complete integration with the main file tree structure and UI.

4 Testing Plan

4.1 Objectives

- Verify correctness of Scanning, Text View, Tool Bar, Graphical View, and DiskOver features.

- Verify cross-platform behavior for Windows and macOS executables built with Qt Creator.
- Verify stability, performance, and safety (cancellation, delete/rename flows).

4.2 Test Environment and Build Notes

Windows: Built with Qt Creator (Win10/11).

macOS: Built with Qt Creator (macOS 12+).

- On Windows, `moveToRecycleBin()` uses `SHFileOperationW`. Test long paths, network drives, admin permissions.
- On macOS, `moveToRecycleBin()` uses `osascript` + `Finder`. Verify spaces in paths, sandboxing, and permissions.

4.3 Test Data

- **Tiny:** 0–5 files (edge case)
- **Typical:** 100–1,000 files (mixed types)
- **Large:** 10k–100k files, nesting depth > 15
- **Special:** Hidden files, symbolic links, permission-denied folders, long paths (> 260 chars, Windows)
- **DiskOver Datasets:** Duplicates, temporary files, and mixed multimedia/doc/archive types for visualization.

4.4 Detailed Test Cases

Each test case includes: ID, Title, Preconditions, Steps, Expected Result.

4.4.1 A. Scanning (SCN-xxx)

SCN-001	Scan Empty Folder Pre: Empty folder exists. Steps: Click <i>Scan</i> → select empty folder. Expected: Progress = 100%, tree shows only root, no crash.
SCN-002	Scan Small Folder Steps: Scan folder with 10 files. Expected: Progress updates incrementally, rootNode.size = sum of file sizes.
SCN-003	Scan Large Folder (Performance) Pre: Dataset with > 10k files. Steps: Start scan. Expected: UI remains responsive barring initial file counting required for progress bar (uses QApplication::processEvents()), correct item count.
SCN-004	Cancel Scan Midway Steps: Click <i>Cancel</i> during large scan. Expected: Scan stops gracefully, progress reset, no hanging threads.
SCN-005	CountTotalItems correctness Steps: Compare actual file system count vs displayed count. Expected: Totals match.
SCN-006	Re-Scan After Changes Steps: Modify files → click <i>Re-scan</i> . Expected: Updated directory contents shown.
SCN-007	Boundary: Deep Nesting Path Steps: Scan directory depth > 20. Expected: Handles recursion without stack overflow; sizes correct.
SCN-008	Negative: Permission-Denied Folders Steps: Include protected folders. Expected: Opening the file/folder based on permissions.

4.4.2 B. Tool Bar (TBR-xxx)

TBR-001	Display Format Units Steps: Change display to KB/MB/GB/Bytes. Expected: Values update correctly, raw bytes preserved.
TBR-002	Search Box Steps: Search multiple matches. Expected: Matched items highlighted, tree expanded on matches.
TBR-003	Filter by Size Steps: Apply <i>Greater than X MB</i> or <i>Less than X MB</i> (filter based on current unit). Expected: Filtered correctly.
TBR-004	Reset Filter Steps: Apply filters → Reset. Expected: All items visible again.
TBR-005	Font Size Change Steps: Increase font size to 30. Expected: Font updates without clipping or layout break.
TBR-006	Export CSV / JSON / PDF Steps: Export all. Expected: File created with correct contents (CSV rows, JSON structure, PDF with table). Validate a sample file.
TBR-007	Boundary: Very Large Export Pre: 100k+ items. Expected: Export completes within timeout; memory use reasonable.

4.4.3 C. Graphical View (GPH-xxx)

GPH-001	Toggle Graphical View Expected: TreeMap visible toggles correctly.
GPH-002	Drill-down by Click Expected: Correct root change with colouring according to legend.
GPH-003	Squarify Layout Correctness Expected: No overlap, proportional area, and text if present is readable. If not present, can get metadata on hovering.
GPH-004	Max Depth Menu Expected: Updates and reflects visually.
GPH-005	Boundary: Deep Tree Visualization Steps: Render folder depth > 10. Expected: Maintains usability and performance.
GPH-006	Negative: Null Data Input Steps: Attempt to render after scanning empty directory. Expected: No crash and no graphical view visible.

4.4.4 D. DiskOver / Free Space (DSK-xxx)

DSK-001	FileType Pie Chart Expected: pie chart dialog appears; slice totals sum to rootNode.size (or close). Pie chart proportion of images, documents, audio, code, etc. should match percent in the parent folder (manually check for a created folder/file).
DSK-002	Smart Duplicate Detection Pre: Two identical files: same size, same content. Two files with same size, different content (less than 2 MB). Two files with same size, different content (greater than 2 MB) - to ensure hashing takes place as expected in small files as well. Two files with different size, different content Expected: Identical files grouped correctly.
DSK-003	Smart Deletion Suggestions Expected: Sensible scoring and ranking of large/old/tmp files.
DSK-004	Delete to Recycle/Trash Expected: File moved to Recycle Bin/Trash; refresh updates view.
DSK-005	Boundary: Extremely Large Files (> 4GB) Expected: Hashing and deletion stable as hashing only of start and end 1 MB

4.5 Execution Priorities

- **P0 (Critical):** Scanning, cancel, delete, export, crash-free across OS.
- **P1 (Important):** Graphical layout, duplicate detection, pie charts, filters.
- **P2 (Optional):** Performance timing, cross-platform execution.

5 Further Improvements

1. Enhance the graphical interface by introducing visually appealing icons to make the application more intuitive and aesthetically pleasing.
This can be done by doing more lit survey and trying out more optimizing algorithms to get better views. We tried Slice and Dice and the squarified algorithm, but there are more options we could explore for the same.
2. Further optimize the tree map visualization algorithm to handle large directory structures more efficiently and reduce rendering lag.
We can use multiple threads to run the UI and the backend separately so the frontend does not lag while scanning and rendering.
3. Implement a caching mechanism to store data from previous scans, allowing repeated scans of the same directory to load much faster without fully reprocessing the filesystem in the same run (Assuming no changes have been made. Put a disclaimer for this to the user.)

This can be implemented by storing the Filenode tree in memory. If the user revisits the same directory, use the cached data to make the process faster and display the previous results directly. Note that this assumes that changes have not been made. This would be a useful feature if the user is switching between scannign directories in the same run without making any changes.

4. Introduce search history, so the user does not have to re navigate to directpries that they recently scanned.
This can be implemented by caching the paths of the directories that have been recently scanned.
5. Extend cross-platform compatibility by making the application fully operational on Linux systems in addition to Windows and MacOS.
6. Refine and expand the clear storage options (DiskOver) to provide users with more efficient and customizable ways to manage unused or redundant files.
This was just a heuristic. We could improve this by considering hashing larger portions to get better results.
7. Ensure that there is no stack overflow for very deep directories by considering an iterative scan or track depth.
8. Follow a more thorough test plan to deal with more minute edge cases such as ensuring that files renamed/deleted during scan are considered nonexistent file at populate time and are handled appropriately.

6 References

- Prof.PPD Lecture Slides
- Qt Documentation <https://doc.qt.io/qt-6.9/>
- Bruls, M., Huizing, K., & Van Wijk, J. J. (2000). Squarified Treemaps. In Eurographics (pp. 33–42). https://doi.org/10.1007/978-3-7091-6783-0_4

7 Declarations

1. **Poulomi Sarkar:** No conflict declared. Work was split equally.
2. **Roshni Pai:** No conflict declared. Work was split equally.